

Quadro Vivente: Relazione del Progetto di Laboratorio di Making

Giovanni Maria Savoca
giovannimaria.savoca@studio.unibo.it

10 giugno 2025

Sommario

Il progetto “Quadro Vivente” si propone di trasformare i dati ambientali rilevati in tempo reale in un’esperienza visiva. L’idea centrale è quella di utilizzare un microcontrollore *ESP32* e sensori ambientali per catturare parametri come temperatura, umidità, luminosità e livello sonoro, trasmettendoli a un server *Raspberry Pi* tramite rete Wi-Fi.

Il server elabora le informazioni ricevute da tutti i client connessi, aggiornando una rappresentazione grafica sotto forma di “quadro vivente”. Questa animazione riflette le condizioni dell’ambiente circostante.

Il tutto è ospitato in una struttura progettata su misura, con sensori saldati a mano su PCB e racchiusi in una scatola stampata in 3D. Il progetto si presta a future evoluzioni in ambito smart-art, installazioni ambientali e monitoraggio interattivo.

1 Introduzione

Il progetto “Quadro Vivente” nasce dall’idea di trasformare dati ambientali in un’esperienza visiva dinamica. Utilizzando sensori di *DHT11* (temperatura e umidità), *LDR* (luce) e *MAX9814* (suono), il sistema genera in tempo reale immagini naturalistiche che variano in base alle condizioni rilevate.

L’ispirazione è arrivata riflettendo su come i sensori ambientali potessero dare vita a un’entità quasi “viva”: inizialmente avevo pensato di tradurre le variazioni in una colonna sonora in continua evoluzione, generando rumori e suoni ambientali. Tuttavia, valutando l’impatto sull’utente, ho ritenuto più evocativo realizzare un “quadro” in movimento su uno schermo.

La versione definitiva del progetto prevede inoltre di trasmettere in diretta su YouTube il “Quadro Vivente” per tutta la durata del funzionamento dell’ESP32 e del Raspberry, permettendo a chiunque di osservare l’evolversi delle animazioni in tempo reale.

2 Obiettivi

Con questo progetto ho voluto innanzitutto realizzare un sistema capace di rilevare in modo affidabile i principali parametri ambientali—temperatura, umidità, intensità luminosa e livello sonoro—utilizzando un microcontrollore. Successivamente, l’obiettivo è stato garantire la trasmissione in tempo reale di questi dati a un server dedicato, in modo da trasformare le misure in animazioni grafiche evocative. Ho voluto poi sviluppare un’interfaccia web interattiva, accessibile da più postazioni, che permettesse di osservare in ogni momento il “quadro vivente” aggiornarsi in base alle condizioni rilevate.

Durante lo sviluppo, confrontandomi con il tutor del corso (Ivan), ho compreso che una semplice breadboard non sarebbe stata una soluzione accettabile per la consegna finale del progetto. Per questo motivo ho deciso di acquistare l’attrezzatura necessaria per effettuare saldature in autonomia, insieme a un nuovo modulo *ESP32*, su cui ho saldato direttamente i sensori ambientali.

Questo passaggio è stato fondamentale per poter progettare e stampare in 3D un contenitore su misura, solido e ordinato, capace di ospitare in modo stabile l'elettronica del sistema.

3 Architettura del Sistema

Il sistema si articola in due componenti principali, ciascuna con un ruolo ben definito e progettata per garantire modularità e scalabilità:

Client (ESP32) Il microcontrollore *ESP32* funge da unità di acquisizione dati: legge i valori di temperatura, umidità, luminosità e livello sonoro attraverso i sensori *DHT11*, *LDR* e *MAX9814*. Grazie al modulo Wi-Fi integrato, l'ESP32 formatta i dati in un oggetto JSON e li invia al server tramite una richiesta HTTP POST verso un endpoint dedicato, ad esempio `/dati`.

Server (Raspberry Pi) Il *Raspberry Pi* riceve i dati dal microcontrollore ed elabora le informazioni attraverso un'applicazione Python basata su Flask. I valori ambientali vengono decodificati e utilizzati per aggiornare una struttura dati condivisa, che alimenta la generazione visiva in tempo reale sul browser. La risposta HTTP conferma l'avvenuta ricezione, mentre l'interfaccia grafica collegata al server si aggiorna immediatamente in base ai nuovi dati ricevuti. L'architettura è pensata per essere facilmente espandibile: ulteriori dispositivi client possono essere aggiunti senza modificare la logica principale del server.

Flusso di Dati Il flusso informativo si sviluppa in cinque passaggi principali:

1. Il microcontrollore ESP32 acquisisce i dati dai sensori ambientali: *DHT11*, *LDR* e *MAX9814*.
2. I dati vengono impacchettati in formato JSON.
3. L'ESP32 invia una richiesta HTTP POST al server Flask.
4. Il server riceve e decodifica il messaggio, aggiornando la memoria interna con i nuovi valori.
5. L'interfaccia web collegata, accedendo al server, aggiorna la visualizzazione grafica in tempo reale per riflettere i nuovi parametri ambientali.

Questa separazione tra livello di acquisizione e livello di rappresentazione semplifica l'estensione futura del progetto: è possibile integrare nuovi sensori o modificare il comportamento grafico senza dover intervenire sull'intero sistema.

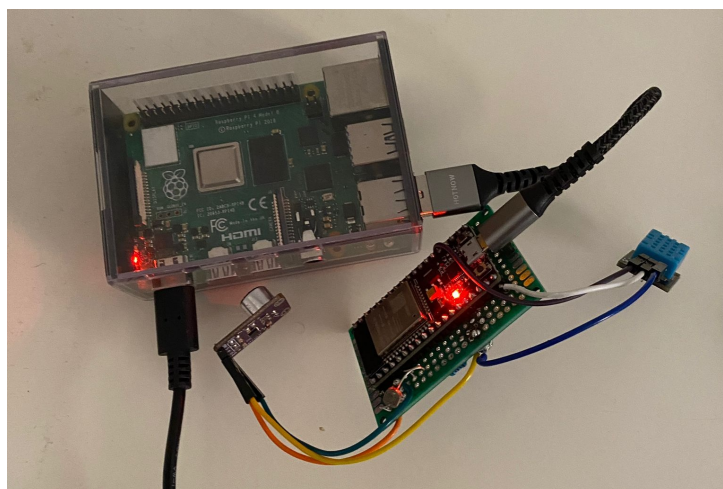


Figura 1: Raspberry Pi con modulo ESP32 e sensori collegati.

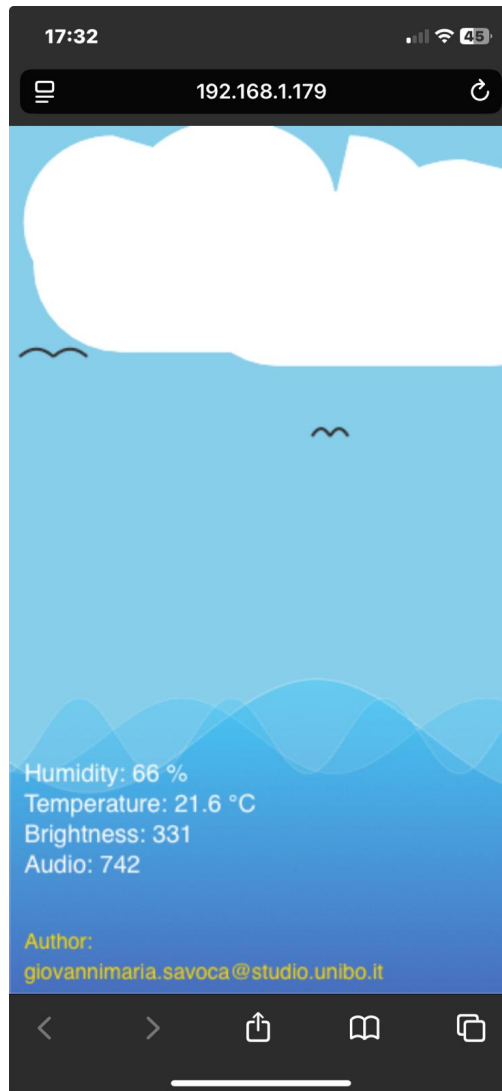


Figura 2: Screenshot del “Quadro Vivente” in esecuzione nel browser da smartphone.

4 Collegamenti dei Sensori

Per la rilevazione dei dati ambientali sono stati utilizzati tre sensori distinti: il microfono analogico MAX9814, una fotoresistenza (LDR) e un sensore DHT11 per temperatura e umidità. I collegamenti sono stati effettuati con attenzione per garantire la compatibilità elettrica e la stabilità del sistema.

4.1 Microfono MAX9814

Il microfono analogico MAX9814 è alimentato a **3.3V**, in linea con le specifiche dell’ESP32. I collegamenti sono i seguenti:

- **VCC** del sensore collegato al pin 3.3V della scheda ESP32.
- **GND** collegato al pin di massa **GND**.
- Il segnale analogico **OUT** collegato al pin **GPIO36 (ADC1_CH0)**.

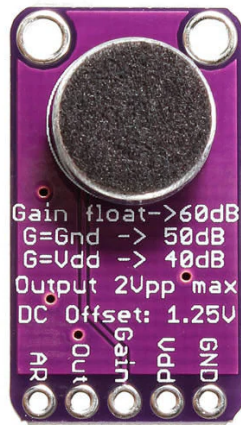


Figura 3: Sensore MAX9814 (microfono analogico con AGC).

4.2 Fotoresistenza (LDR)

La fotoresistenza *LDR* è collegata in serie a una resistenza da **10 k Ω** , in modo da ottenere un segnale analogico proporzionale alla luminosità ambientale. Lo schema di collegamento è il seguente:

- Un'estremità della LDR è connessa al 3.3V.
- L'altra estremità è collegata a un nodo comune con la resistenza.
- L'altro capo della resistenza è collegato a GND.
- Il nodo comune tra LDR e resistenza è collegato al pin analogico GPIO34 (ADC1_CH6), per leggere la variazione di luminosità ambientale.



Figura 4: Fotoresistenza utilizzata come partitore per la lettura della luminosità.

4.3 Sensore DHT11

Il sensore digitale DHT11 fornisce i valori di temperatura e umidità. I collegamenti sono i seguenti:

- **VCC** del sensore collegato al pin 5V dell'ESP32.
- **GND** collegato alla massa comune.
- Il pin dati **DATA** collegato al GPIO4.

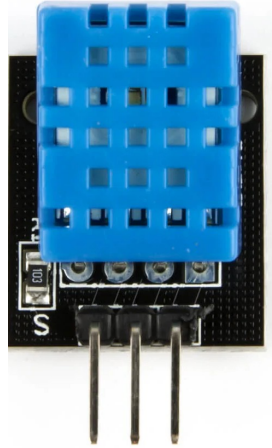


Figura 5: Sensore digitale DHT11 per temperatura e umidità.

Questa configurazione ha permesso di leggere in modo affidabile i dati ambientali con un assorbimento contenuto, garantendo una piena compatibilità con i livelli logici e di alimentazione dell'ESP32.

5 Processo di Sviluppo

All'inizio ho voluto concentrarmi esclusivamente sul funzionamento dei sensori e sulle connessioni elettriche:

1. **Test con Arduino Uno:** ho collegato i tre sensori a un Arduino Uno per verificare la corretta lettura e calibrazione dei valori. Questo passaggio mi ha permesso di familiarizzare con le librerie e di risolvere eventuali problemi di alimentazione e taratura. In parallelo, ho iniziato a progettare la componente visiva del “quadro” utilizzando *Pygame*, realizzando una prima versione dell'animazione in ambiente desktop. Successivamente, per rendere il sistema accessibile via browser, ho convertito tutta la logica grafica in JavaScript.
2. **Passaggio a ESP32:** una volta consolidata la parte di acquisizione, ho adottato un ESP32 per sfruttare il modulo Wi-Fi integrato, semplificando l'invio dei dati al server.
3. **Realizzazione PCB:** ho quindi acquistato l'attrezzatura per la saldatura e ho assemblato tutti i componenti su una scheda a circuito stampato (PCB), assicurandomi della solidità dei collegamenti e dell'assenza di cortocircuiti. Per la prima volta ho effettuato personalmente la saldatura di tutti i componenti, imparando le tecniche base di stagno e fluxante, il corretto riscaldamento dei pad e la prevenzione di cortocircuiti. Questa nuova competenza si è rivelata fondamentale per garantire la robustezza dei collegamenti e mi ha dato maggiore autonomia nel prototipare soluzioni hardware in futuro.
4. **Configurazione del server e comunicazione:** completata la parte hardware, ho configurato il *Raspberry Pi* come server centrale del sistema. Ho sviluppato un'applicazione in Python basata su *Flask*, che riceve i dati dal microcontrollore ESP32 e li rende disponibili all'interfaccia grafica. La comunicazione client-server è stata realizzata tramite richieste HTTP POST, mentre l'aggiornamento in tempo reale della scena animata è gestito attraverso WebSocket.
5. **Verifica finale e struttura fisica:** dopo aver confermato il corretto funzionamento dei sensori saldati, ho progettato un contenitore personalizzato, tenendo conto delle dimensioni della PCB e del posizionamento dei sensori. Una volta completato il modello, ho chiesto

al mio amico Andrea Loretto di realizzare la stampa 3D. Il risultato è una scatola su misura che ospita in modo ordinato tutti i componenti, offrendo protezione e facilitando la manutenzione.

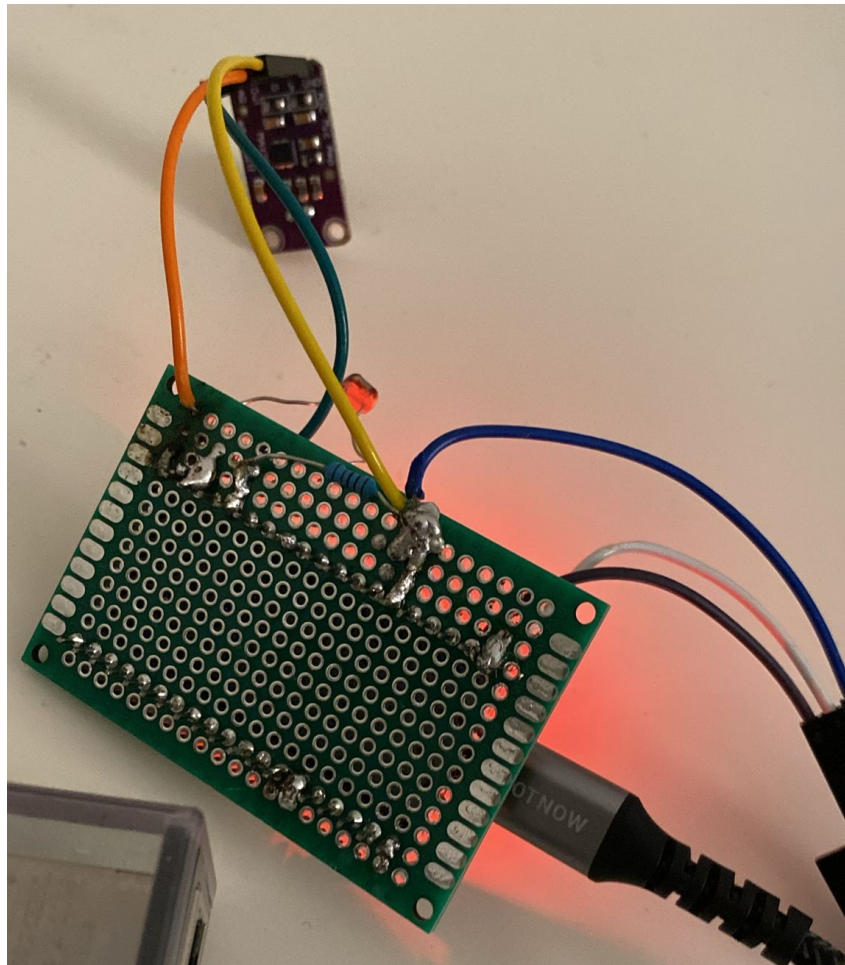


Figura 6: Fase di saldatura dei componenti sulla PCB realizzata.

6 Logica degli Script

In questa sezione descriviamo la struttura della repository e il funzionamento dei due principali script spiegandone la logica operativa.

6.1 Struttura della repository

La cartella principale del progetto, disponibile su GitHub, è organizzata come segue:

```
V1/                                     % script e prototipi preliminari di sviluppo e test
Raspberry/                             % implementazione finale del progetto completo
|-- Arduino/
|   '-- sketch_apr21a.ino              % codice ESP32
|-- static/                            % file JavaScript e CSS
|   |-- main.js
|   '-- style.css
|-- Templates/                         % template HTML per Flask
|   '-- Screen.html
```

```
|-- client_secret.json          % credenziali di accesso YouTube (non committate)
|-- fake_data.py               % simulatore di dati sensoriali
|-- live.py                    % script per streaming su YouTube
|-- requirements.txt           % dipendenze Python
|-- server.py                  % servizio Flask + SocketIO
'-- token.pickle                % credenziali di accesso (non committate)
```

6.2 Codice del client ESP32

Il firmware dell'ESP32 risiede in `Raspberry/Arduino/sketch_apr21a.ino`. All'accensione, il microcontrollore:

- inizializza il modulo Wi-Fi e si connette alla rete specificata;
- acquisisce in loop i dati dai sensori (DHT11, LDR, MAX9814), campionando la forma d'onda del microfono e leggendo temperatura, umidità e luminosità;
- impacchetta le misure in un oggetto JSON e, se il Wi-Fi risulta connesso, effettua una richiesta HTTP POST verso l'endpoint `/dati` del Raspberry Pi;
- invia infine sul monitor seriale sia i valori raw per il Plotter che stringhe annotate per il debug.

6.3 Servizio Python sul Raspberry Pi

Lo script `Raspberry/server.py`, eseguito automaticamente all'avvio del sistema, implementa:

- un server Flask che espone:
 - la rotta `/` per servire `Templates/Screen.html`;
 - la rotta `/dati` in POST per ricevere i JSON dall'ESP32.
- un'istanza Flask-SocketIO che, ad ogni arrivo di nuovi dati, emette un evento **update** verso tutti i client WebSocket connessi;
- gestione degli errori di formato e validazione dei dati ricevuti;
- avvio in ascolto su tutte le interfacce di rete (porta 8000), in modalità debug per facilitare lo sviluppo.

Questa separazione netta tra firmware sul client e servizio Web sul server garantisce modularità. In questo modo, è possibile estendere facilmente il sistema integrando nuovi moduli arduino: sarà sufficiente modificare il codice del server per gestire i nuovi dati e aggiornare la visualizzazione sul sito web, mantenendo invariata l'architettura complessiva.

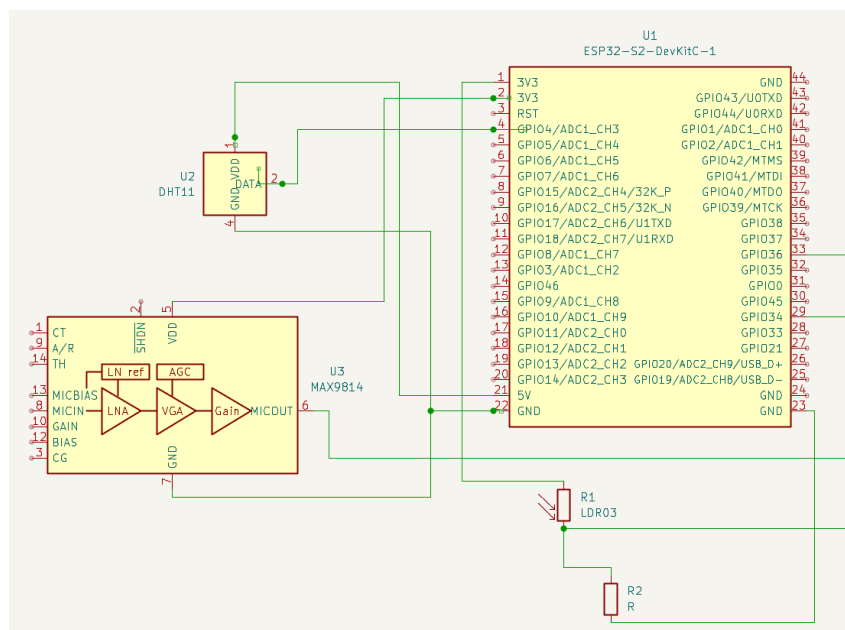


Figura 7: Circuito con KiCad6 dell'esp e i sensori.

7 Valutazione e Prospettive Future

Il progetto “Quadro Vivente” ha dimostrato come l’integrazione di elettronica e tecnologie web possa generare un’esperienza artistica e interattiva basata su dati reali. Durante le sperimentazioni, il sistema si è rivelato affidabile e reattivo: il ciclo completo di acquisizione, invio e aggiornamento grafico si è mantenuto entro i 1000 ms, garantendo un effetto visivo percepito come “live”. Le transizioni cromatiche, come il passaggio dal giorno alla notte, sono risultate graduali e naturali, mentre elementi dinamici come uccellini o foglie hanno arricchito la scena rendendola più espressiva.

In sintesi, il prototipo ha raggiunto gli obiettivi iniziali di modularità, interattività e semplicità d’uso, offrendo una base solida su cui costruire ulteriori evoluzioni.

Tra gli sviluppi futuri possibili si propongono:

1. **Sensori avanzati:** integrazione di rilevatori di CO₂, qualità dell’aria o pressione atmosferica per arricchire ulteriormente il “linguaggio” visivo.
2. **Storage e analisi storica:** archiviazione dei dati in un database (es. InfluxDB o PostgreSQL) per generare statistiche temporali e trend analysis.
3. **Rendering parametrico:** utilizzo di librerie grafiche basate su dati storici per creare animazioni basate su pattern (ad esempio, variazioni stagionali su più giorni).
4. **Machine Learning:** applicazione di modelli di clustering o riconoscimento di pattern sonori/ambientali per far emergere eventi significativi e generare animazioni contestuali.
5. **Streaming online:** pubblicazione del “Quadro Vivente” su piattaforme come YouTube Live, permettendo la fruizione pubblica dell’esperienza in tempo reale da qualsiasi dispositivo.

Questi possibili ampliamenti rendono il progetto non solo un esercizio accademico, ma anche una base per installazioni artistiche interattive o sistemi di monitoraggio ambientale ad uso educativo, espositivo o domestico.

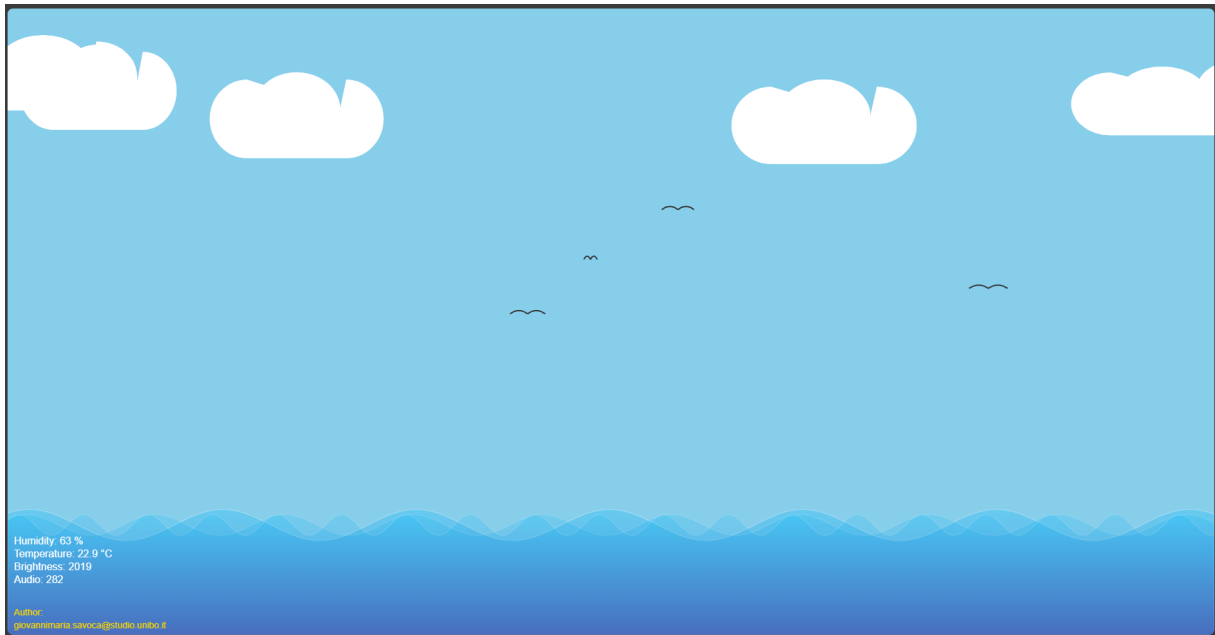


Figura 8: Screenshot del “Quadro Vivente” in esecuzione nel browser.

Riferimenti bibliografici

- [1] Espressif Systems,
ESP32 Series Technical Reference Manual, 2020,
https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [2] Aosong Electronics Co.,
DHT11 Temperature and Humidity Sensor Datasheet,
<https://www.tme.eu/Document/7a4fd48d400b8c4c8309ef1e2b13cdd4/MR003-005-1.pdf>
- [3] Analog Devices,
MAX9814: Microphone Amplifier with AGC and Low-Noise Microphone Bias, 2014,
<https://www.analog.com/media/en/technical-documentation/data-sheets/max9814.pdf>
- [4] Flask,
Flask Web Framework Documentation,
<https://flask.palletsprojects.com>
- [5] Pygame Community,
Pygame Documentation,
<https://www.pygame.org/docs/>
- [6] Davoli, F. e Montori, F.,
Slide dei corsi di Laboratorio di Making,
Università di Bologna, 2025. Materiale didattico disponibile su Virtuale: <https://virtuale.unibo.it/course/view.php?id=57233>